

Lecture 10

Introduction to Machine Learning

1 What is Machine Learning?

2 A Controversy in the Wine World

Classic Artificial Intelligence

Classic AI attempts to codify the rules that a human would use to make decisions.

Example: If you are trying to build a system that finds all the proper nouns in a text document, you might hard-code the following rules:

- If a word is a proper noun, then the first letter of the word is capitalized.
- The first letter of a sentence is always capitalized.
- ...

The system can deduce new rules from existing rules, e.g.,

- It is impossible to tell whether the first word of a sentence is a proper noun just from the capitalization.

Pros: The model is super interpretable!

Cons: For complex tasks, there are too many rules, and we can't anticipate them all

What is Machine Learning?

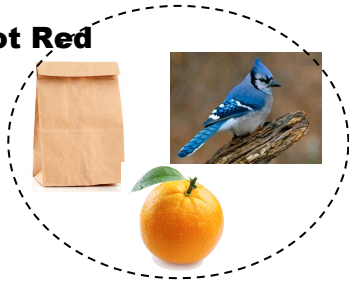
Exercise: Pair up with the person sitting next to you. One of you will be an Earthling, the other a Martian.

The Earthling should explain to the Martian what the word “red” means. The Martian should try to be obtuse.

Moral: We often learn by seeing examples.



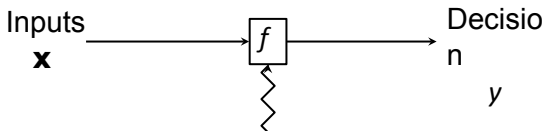
Not Red



Rather than trying to come up with the rules ourselves, we can learn the rules from data. This is the essence of **machine learning**.

What is Machine Learning?

Learning refers to the act of coming up with a rule for making decisions based on a set of inputs.

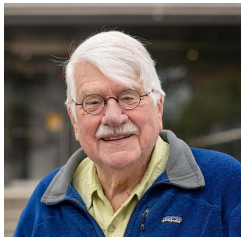


Goal of Machine Learning: Come up with a rule f from **training data** ($\mathbf{x}_i, y_i$).

The decision y is typically called the **target** or the **label**.

1 What is Machine Learning?

2 A Controversy in the Wine World



Orley Ashenfelter
Economics
Professor

In 1991, Orley Ashenfelter predicted that the 1986 vintage of Bordeaux wines would be disappointing.

He did this without tasting a drop of the wine.



Robert Parker
*The Wine
Advocate*

Wine critics were outraged.

Robert Parker had predicted that the 1986 vintage would be “very good and some- times exceptional” based on tasting an early sample.

How did Ashenfelter make this prediction?

Ashenfelter collected data on summer temperature and winter rainfall in Bordeaux from 1952 to 1991.

The quality of wines becomes apparent after 10 years. So for vintages up to 1980, he also collected their price.

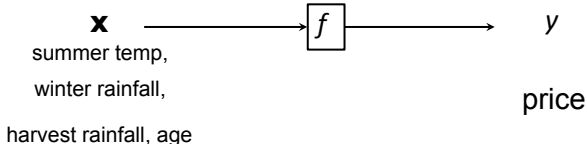
```
import pandas as pd
```

```
df = pd.read_csv("https://dlsun.github.io/pods/data/bordeaux.csv",  
                 index_col="year")
```

```
df
```

	price	summer	har	sep	win	age
year						
1952	37.0	17.1	160	14.3	600	40
1953	63.0	16.7	80	17.3	690	39
1955	45.0	17.1	130	16.8	502	37
1957	22.0	16.1	110	16.2	420	35
...	...	...	...	...	...	...
1988	NaN	17.1	59	16.8	808	4
1989	NaN	18.6	82	18.4	443	3
1990	NaN	18.7	80	19.3	468	2
1991	NaN	17.7	183	20.4	570	1

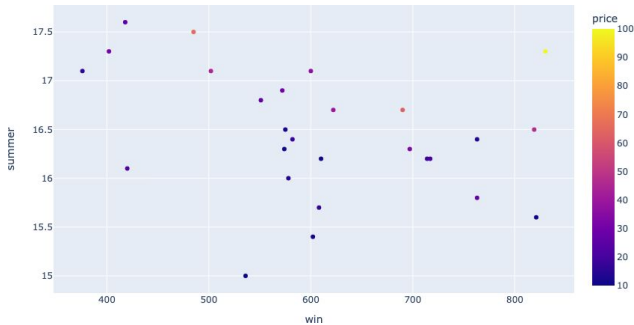
38 rows × 6 columns



Visualizing the Data

```
import plotly.express as px
```

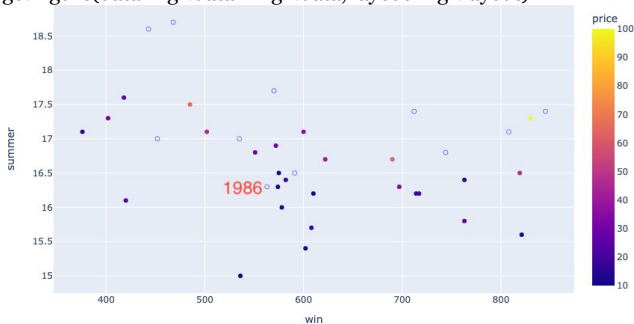
```
px.scatter(df[~df["price"].isnull()],  
           x="win", y="summer", color="price")
```



Visualizing the Data

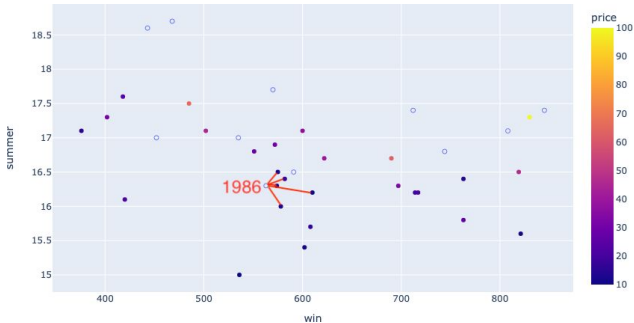
```
import plotly.graph_objects as go
```

```
fig1 = px.scatter(df[~df["price"].isnull()],  
                  x="win", y="summer", color="price") fig2  
= px.scatter(df[df["price"].isnull()],  
             x="win", y="summer", symbol_sequence=["circle-open"])  
go.Figure(data=fig1.data + fig2.data, layout=fig1.layout)
```



What would you predict is the quality of the 1986 wine?

Visualizing the Data



Insight: The “closest” wines are low quality, so the 1986 vintage is probably low quality as well.

This is the intuition behind *k*-**nearest neighbors**.

<https://colab.research.google.com/drive/1ASh4m45-PiMe8u6dSyl-xjCNsq7vP5y>

Types of Machine Learning Problems

- Machine learning problems are grouped into two types, based on the type of y :

- **Regression:** The label y is quantitative.

- **Classification:** The label y is categorical.

- Was Ashenfelter's wine problem a regression or a classification problem?

- Note that the input features $\mathbf{x}$ may be categorical, quantitative, textual, ..., or any combination of these.

***K*-Nearest Neighbors**

https://colab.research.google.com/drive/1vhEyKnRYfnnvuc7bOzBr_8Ex2vF3_2ew1

K-Nearest Neighbors

The data for which we know the label y is called the **training data**.

The data for which we don't know y (and want to predict it) is called the **test data**

```
df_train = df.loc[:1980].copy() df_test  
= df.loc[1981:].copy()
```

K-Nearest Neighbors

- 1 For each observation in the test data, find the k “nearest” observations in the training data based on input features $\mathbf{x}$ (e.g., summer temperature and winter rainfall).
- To predict the label y (e.g., price) for the test observation, average the labels of these k “nearest” training observations.

K-Nearest Neighbors from Scratch

Before computing distances, we should scale the

```
√ X_train = df_train[["win", "summer"]]  
y_train = df_train["price"]
```

```
# Standardize the features.
```

```
X_train_mean = X_train.mean()
```

```
X_train_sd = X_train.std()
```

```
X_train_scaled = (X_train - X_train_mean) / X_train_sd
```

We should scale the test data in exactly same

```
X_test = df_test[["win", "summer"]]
```

```
X_test_scaled = (X_test - X_train_mean) / X_train_sd
```

```
X_test_scaled
```

```
      win  summer  
year  
1981 -0.568896  0.812183  
1982  0.802826  1.425581  
1983  1.833554  1.425581  
1984 -0.134905  0.045437  
...      ...      ...  
1988  1.546810  0.965533  
1989 -1.281881  3.265772  
1990 -1.088135  3.419122  
1991 -0.297651  1.885629
```

11 rows x 2 columns



K-Nearest Neighbors from Scratch

Next, we calculate the (Euclidean) distances between a vintage in the test set and the vintages in the training data.

```
import numpy as np
dists = np.sqrt(
    ((X_test_scaled.loc[1986] - X_train_scaled) ** 2).sum(axis=1))
dists
year
1952    1.259860
1953    1.159726
1955    1.314727
1957    1.149883
...
1977    2.269387
1978    1.729248
Length: 27, dtype: float64
1979    1.263287
1980    0.474508
```

Now we just need to sort these distances and take the first $k = 5$.

```
index_nearest = dists.sort_values().index[:5]
index_nearest
```

```
Int64Index([1974, 1958, 1969, 1968, 1980], dtype='int64', name='year')
```



K-Nearest Neighbors from Scratch

Finally, to make a prediction, we average the labels y of these

```
y_train[index_nearest].mean()
```

13.2

That's \$13.20 for a bottle of wine. So 1986 is not a good

vintage. How do we do this for every vintage in the test set?

K-Nearest Neighbors in Scikit-Learn

Scikit-learn provides a built-in model `KNeighborsRegressor` that fits k -nearest neighbors regression models.

But first, we need to scale the training and test data.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
scaler.fit(X_train)
```

```
X_train_scaled = scaler.transform(X_train)
```

```
# Scale the test data using a scaler that was fit to the training data! X_test_scaled =  
scaler.transform(X_test)
```

Now, we fit k -nearest neighbors using `KNeighborsRegressor`.

```
from sklearn.neighbors import KNeighborsRegressor
```

```
model = KNeighborsRegressor(n_neighbors=5)
```

```
model.fit(X=X_train_scaled, y=y_train)
```

```
model.predict(X=X_test_scaled)
```

```
array([35.8, 54. , 52.2, 18.4, 35.6, 13.2, 37. , 51.4, 36.6, 36.6, 40.6])
```

Pipelines in Scikit-Learn

In the code above, we had to be careful to standardize the training and test data in exactly the same way.

Machine learning models typically involve many more preprocessing steps.

Scikit-Learn's Pipeline allows us to chain steps together.

```
from sklearn.pipeline import make_pipeline

pipeline = make_pipeline(
    StandardScaler(),
    KNeighborsRegressor(n_neighbors=5))
```

We can use this Pipeline like any other machine learning

```
pipeline.fit(X=X_train, y=y_train)
pipeline.predict(X=X_test)
```

```
array([35.8, 54. , 52.2, 18.4, 35.6, 13.2, 37. , 51.4, 36.6, 36.6, 40.6])
```



Was Ashenfelter Right?

Here is how the vintages rank

Vintage	Score	Drink Window	Description
1994	85	Drink	Medium-bodied, with good fruit and firm tannins
1993	82	Past peak	Good color, perfumy, fruity and balanced
1992	72	Past peak	Light, simple and often diluted; early maturing
1991	72	Past peak	Lean, tough and light; stick to top names
1990	97	Drink	Opulent, well-structured and harmonious
1989	98	Drink	Bold, dramatic fruit character; tannic and long-aging
1988	93	Drink	Racy, fruity wines, with firm tannins and typical structure; best in Pessac-Léognan, Pomerol and Pauillac
1987	76	Past peak	Delicate, ripe yet diluted
1986	95	Drink	Powerful, intense and tannic; best in Médoc
1985	93	Drink	Balanced, supple and fruity; defines finesse
1984	70	Past peak	Unripe, astringent and dry

Ashenfelter was wrong that 1986 would be a
disappointing vintage!



Prediction Error

If the true labels are $y_1, \dots, y_n$ and our model predicts $\hat{y}_1, \dots, \hat{y}_n$, how do we measure how well our model did?

- **mean squared error (MSE)**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **mean absolute error (MAE)**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Calculating MSE or MAE requires data where true labels are known. Where can we find such data?

<https://colab.research.google.com/drive/1ZQMPuO2dNEB75SDsDdothSRaUjBZGHyb>



Training

Error

On the training data, the true labels $y_1, \dots, y_n$ are known.

```
pipeline.fit(X_train, y_train) y_train_ =  
pipeline.predict(X_train) ((y_train -  
y_train_) ** 2).mean()
```

207.24148148148146

There's also a Scikit-Learn function for

```
from sklearn.metrics import mean_squared_error  
mean_squared_error(y_train, y_train_)
```

207.24148148148146

How do we interpret this MSE of 207.24?

Remember, we are predicting the price of wine. So the model is off by 207.24 square dollars on average.

The square root is easier to interpret. The model is off by $\sqrt{207.24} \approx \$14.40$ on average. This is called the **RMSE**.

The Problem with Training Error

What's the training error of a 1-nearest neighbor model?

```
pipeline = make_pipeline(  
    StandardScaler(),  
    KNeighborsRegressor(n_neighbors=1))  
pipeline.fit(X=X_train, y=y_train)  
y_train_ = pipeline.predict(X=X_train)  
mean_squared_error(y_train, y_train_)  
0.0
```

Why did this happen?

The 1-nearest neighbor to any observation in the training data is the observation itself!

A 1-nearest neighbor model will always be perfect on the training data. But is it necessarily the best model?



Test Error

We don't need to know how well our model does on *training data*. We want to know how well it will do on *test data*.

In general, test error > training error.

Analogy: A professor posts a practice exam before an exam.

- If the actual exam is the same as the practice exam, how many points will students miss? That's training error.
- If the actual exam is different from the practice exam, how many points will students miss? That's test error.

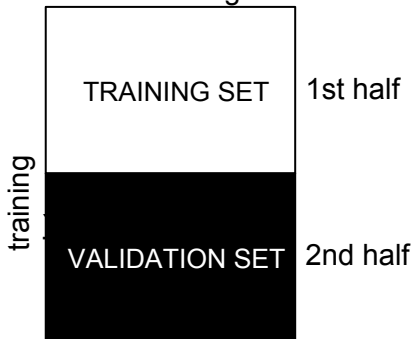
It's always easier to answer questions that you've seen before than questions you haven't seen.

Now: How do we estimate the test error?

Validation Set

The training data is the only data we have, where the true labels y are known.

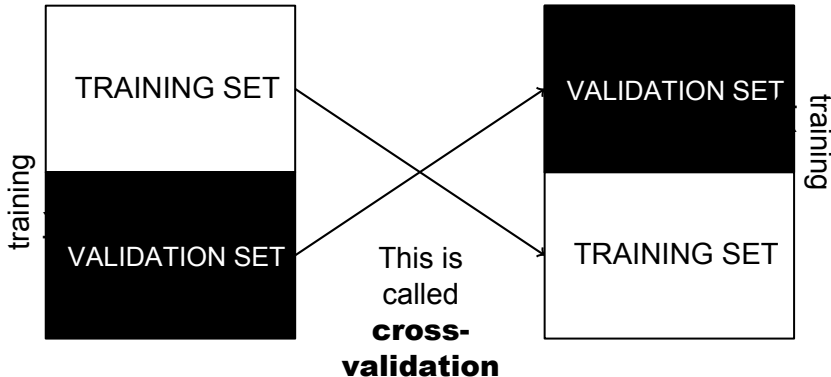
So one way to estimate the test error is to not use all of the training data to fit the model, leaving the remaining data for estimating the test error.



Cross-Validation

The way we split the data into two halves was arbitrary.

Why not use the 2nd half for training and the 1st half for validation?



Implementing Cross-Validation from Scratch

Previously, we fit the model to the training set and evaluated the predictions on the validation set.

```
pipeline.fit(X_train_set, y_train_set) y_val_set_ =  
pipeline.predict(X_val_set)  
mean_squared_error(y_val_set, y_val_set_)
```

195.71428571428572

Now let's do the same thing, but with the roles of the training and validation sets reversed.

```
pipeline.fit(X_val_set, y_val_set) y_train_set_ =  
pipeline.predict(X_train_set)  
mean_squared_error(y_train_set, y_train_set_)
```

306.9230769230769

Wow, the estimates can be quite different!

To come up with one overall estimate of the test error, we can average them.

```
(195.71 + 306.92) / 2
```

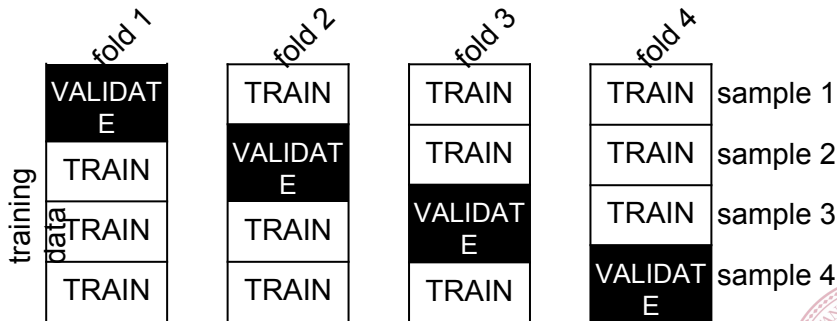
251.315

K-Fold Cross Validation

One problem with splitting the data into two is that we only fit the model on half of the data.

A model trained on half of the data may be very different from a model trained on all of the data.

It may be better to split the data into K samples and come up with K validation errors.



This way, we use $1 - 1/K$ of the data for training.



Implementing Cross-Validation in Scikit-Learn

You specify the model, data, and K . Scikit-Learn will:

- split the training data into K samples
- hold out one sample at a time as a validation set
- fit the model to remaining $1 - 1/K$ of the data

```
from sklearn.model_selection import cross_val_score scores =  
cross_val_score(  
    pipeline,  
    X=df_train[["win", "summer"]],  
    y=df_train["price"],      # this is all of the training data!  
    scoring="neg_mean_squared_error", # higher is better for a score cv=4)  
scores  
array([-547.          , -405.85714286, -67.          , -31.          ])
```

So an overall estimate of test MSE is:

```
-scores.mean()  
262.714285714285
```